27/9/24

Experiment-6

ERROR SELECTION AND ERROR CORRECTION

AIM:

Write a Pgm to implement error detection and correction using hamming code concept - Make a test run to input data stream and verify error correction feature.

Sender Pgm:

1) Input to sender file should be a text of any length pgm should convert text to binary

2) Apply hamming concept on the the binary data & add redundant bits to it.

3) Save in a file called channel

Code: (sender.py)

```
def
data = list(map(int,input("Enter data bits:").strip()))
m = len(data)
r = 0
while (2**r) < (m+r+1)
    r += 1
n = m+r
hamming = [0]*(n+1)
j = 0
for i in range(1,n+1):
    if (i & (i-1)) != 0
        hamming[i] = data[j]
        j += 1
for p in range(r):
    pos = 2**p
```

```
parity = 0
for i in range (1, n+1):
    if i & pos:
        parity ^= hamming[i]
hamming[pos] = parity
code = hamming[1:]
print ("Data bits:", data)
print ("hamming code:", code)
```

O/P:

Enter data bits: 1011

Data bit: [1,0,1,1]

Hamming code: [0,1,1,0,0,1,1]

Receiver code: (receiver.py)

```
received = list (map (int, input ("Enter received hamming
code:") strip ()))
n = len (received)
r = 0
while (2**r) != n
    r += 1
error_pos = 0
for p in range (r):
    pos = 2**p
    parity = 0
    for i in range (1, n+1):
        if i & pos:
            parity ^= received[i-1]
    if parity != 0:
        error_pos += pos
```

```
if error_pos == 0:
    print("No error detected")
else:
    print("Error detected at position:", error_pos)
    received[error_pos - 1] ^= 1
    print("Corrected hamming code:", received)
data = []
for i in range(1, n+1):
    if (i & (i-1)) != 0:
        data.append(received[i-1])
print("Original data bits:", data)
```

O/P

Suppose, hc : 0110011 change to 0111011

Error detected at position : 4

Corrected Hamming code : [0, 1, 1, 0, 0, 1, 1]

Original data bits : [1, 0, 1, 1]

RESULT:

Thus the hamming code algorithm was successfully implemented in Python. The sender generated hamming code from given data bits and receiver successfully detected and corrected a single bit error.